



UNIVERSIDADE ESTADUAL DO AMAZONAS  
ESCOLA SUPERIOR DE TECNOLOGIA  
CIÊNCIA DE DADOS - EQUIPE 6

Pedro Yutaro Mont Morency Nakamura  
Alexandre do Valle Ferreira  
Alice Sophia Lima Vieira

**Relatório de Reflexão - Atividade Prática de AutoML**

Manaus  
2026/06'

## CONTEXTO DE ALTERAÇÕES NO DATASET:

### Problema Enfrentado:

Em relação à documentação inicial (Partes 1 e 2), a equipe precisou realizar alterações estruturais significativas no escopo do projeto de Machine Learning após as rodadas iniciais de Análise Exploratória de Dados (EDA). Originalmente, o projeto fora desenhado como uma tarefa de regressão para prever a variável contínua ***stress\_level***. Contudo, a análise da matriz de correlação de Pearson e a visualização de dispersão demonstraram uma ausência quase total de correlação linear entre os atributos preditores (como horas de sono e uso de redes sociais) e os níveis de estresse ou ansiedade. O treinamento de modelos sob essa configuração resultou em métricas de  $R^2$  negativas ou nulas, evidenciando que a tentativa de predição em uma escala contínua de 1 a 10 colapsava para a média, inviabilizando a extração de inteligência do modelo.

A solução proposta é a aplicação de questionários para mapear e categorizar os principais desafios lógicos percebidos pelos estudantes (por exemplo: laços, condicionais, abstração), com o objetivo de gerar um diagnóstico que possa embasar futuras intervenções pedagógicas e aprimorar a formação de novos profissionais na região.

### Solução:

Diante da ausência de sinal matemático forte para regressão, a equipe alterou a natureza do problema para Classificação Binária, elegendo o `depression_label` como a nova variável-alvo, alinhando-se ao objetivo geral do dataset de prever riscos à saúde mental. Para contornar o severo desbalanceamento de classes (onde casos positivos representavam cerca de 2,5% da base), foi aplicada a técnica de sobreamostragem SMOTE (Synthetic Minority Over-sampling Technique) exclusivamente no conjunto de treinamento.

Adicionalmente, para evitar o efeito "Dummy Classifier" — em que o modelo chuta a classe majoritária para minimizar o erro global devido ao limiar padrão de 50% de probabilidade —, a equipe implementou pesos de classe dinâmicos e uma técnica avançada de Threshold Tuning (Ajuste de Limiar). Isso permitiu focar a otimização no F1-Score da classe minoritária, garantindo um modelo funcional para triagem de risco.

## **PARTE 5 - REFLEXÃO**

### **1. Qual ferramenta foi mais fácil de usar? Por quê?**

A segunda ferramenta (**mljar-supervised**) foi significativamente mais fácil de usar. O FLAML, embora extremamente rápido, exigiu uma intervenção manual profunda da equipe para lidar com o desbalanceamento das classes, como o cálculo e a injeção manual de pesos (`sample_weights`) no método de treinamento, além de configurações manuais de estratégias de treinamento (holdout vs. Cross-Validation).

O MLJAR, por outro lado, abstraiu essas complexidades. Ao ser configurado no modo "Perform", ele lidou nativamente com os dados previamente balanceados pelo SMOTE e construiu pipelines de Ensemble e validação cruzada de forma autônoma, exigindo menos linhas de código estrutural por parte dos desenvolvedores.

### **2. Qual ferramenta apresentou melhor desempenho?**

O MLJAR apresentou o melhor desempenho estatístico e prático. Após a aplicação do ajuste de limiar (Threshold Tuning), ambas as ferramentas atingiram o objetivo clínico principal: um Recall de 1,00 (identificando 100% dos alunos com depressão no conjunto de teste). No entanto, o MLJAR foi mais eficiente na separação das classes, gerando apenas 24 Falsos Positivos contra 31 do FLAML. Isso garantiu ao MLJAR um F1-Score superior (0,33 contra 0,28) e uma melhor Área Sob a Curva ROC (0,9274 contra 0,9028).

### **3. A ferramenta com melhor desempenho também foi a mais fácil de usar?**

Sim. O MLJAR entregou o melhor modelo (um Ensemble focado no LightGBM) e a melhor usabilidade, unindo alto poder preditivo com uma interface de código mais amigável. O único contraponto a essa facilidade foi o tempo de execução: a busca detalhada do MLJAR consumiu quase a totalidade do orçamento de tempo (308 segundos), enquanto o FLAML encerrou sua busca heurística em frações de segundo (0,33 segundos).

#### 4. Alguma ferramenta gerou explicações sobre os resultados?

Sim, este foi o grande diferencial do MLJAR. A ferramenta possui um módulo nativo de Explainable AI (**XAI**). Ao final do treinamento, o MLJAR gerou automaticamente um diretório contendo relatórios em Markdown e gráficos prontos, como o mapa de calor de importância de atributos (**features\_heatmap\_scaled**).

Esses relatórios deixaram claro, sem necessidade de código adicional, que as variáveis **sleep\_hours** (0,82) e **daily\_social\_media\_hours** (0,81) foram os fatores determinantes para as previsões do modelo **LightGBM**. O FLAML exigiu a escrita de rotinas personalizadas usando bibliotecas gráficas externas (**Seaborn/Matplotlib**) para extrair o mesmo nível de informação.

#### 5. O AutoML ajudou a reduzir o tempo de desenvolvimento do experimento?

Ajudou significativamente apenas nas etapas de seleção de algoritmos e otimização de hiperparâmetros (*Tuning*). Testar manualmente algoritmos como Random Forest, XGBoost, CatBoost e LightGBM, configurando grades de parâmetros complexas para cada um, levaria horas ou dias. As ferramentas AutoML realizaram essa varredura no espaço de busca em questão de minutos, permitindo que a equipe focasse na resolução do problema de negócio.

Porém, nas etapas de planejamento do dataset e formulação do tipo de problema (classificação vs. regressão), a equipe encontrou grandes dificuldades na conciliação dessa parte em detrimento das automações providenciadas pelas ferramentas AutoML, uma vez que no domínio do dataset não há um atributo alvo bem definido (explicado na publicação do Kaggle). Além da falta de rigor metodológico no agrupamento dos dados apresentados, evidenciada pela escassez de correlações entre atributos preditores e possíveis variáveis alvo no notebook de pré-processamento (diga-se de passagem que esta foi a etapa mais custosa do trabalho inteiro).

## **6. Quais decisões ainda precisaram ser tomadas pela equipe?**

A intervenção analítica da equipe foi o que salvou o experimento de um fracasso metodológico. O AutoML automatiza a busca, mas não compreende o contexto. A equipe precisou:

- Pivotar a variável-alvo de stress\_level (regressão) para depression\_label (classificação) devido à ausência de correlação linear nos dados originais.
- Identificar e mitigar o vazamento de dados (data leakage), removendo variáveis colineares óbvias (anxiety\_level e stress\_level) dos preditores.
- Aplicar SMOTE na camada de processamento (Silver) para corrigir o desbalanceamento severo da base.
- Implementar e testar configurações de treinamento específicas (de acordo com o desbalanceamento de classes) e intervir no ponto de corte da probabilidade (Threshold Tuning), baixando os limiares de 0,50 para 0,18 (FLAML) e 0,08 (MLJAR), assumindo que, no contexto de saúde mental, evitar Falsos Negativos é mais importante do que manter a precisão global.

## **7. Quais problemas poderiam ocorrer se alguém usasse AutoML sem entender Ciência de Dados?**

Visto que o volume e a qualidade dos dados eram baixos, muitos. O usuário incorreria na armadilha do Classificador Ingênuo (Dummy Classifier). Se o experimento tivesse sido rodado sem o tratamento de dados e sem a mudança de métricas da equipe, as ferramentas teriam gerado modelos que simplesmente chutavam "Sem Depressão" para 100% dos casos.

O usuário inexperiente comemoraria uma "Acurácia de 96%" (já que 97% da base era saudável), entregando um modelo cego, totalmente inútil no mundo real e clinicamente perigoso, acreditando tratar-se de um sucesso devido à falta de letramento em métricas adequadas para dados desbalanceados (como F1-Score e Recall).

## **8. Em que situações o AutoML pode ser útil em projetos reais?**

É extremamente útil para a criação rápida de protótipos (Rapid Prototyping) e estabelecimento de baselines (marcos de referência de desempenho). Em cenários corporativos onde o tempo de entrega é curto, o AutoML pode validar rapidamente se um conjunto de dados possui "sinal" suficiente para justificar o investimento de tempo em uma modelagem manual mais profunda.

## **9. Em que situações o AutoML pode não ser adequado?**

Não é recomendado para ambientes com altíssima restrição de memória ou poder computacional (computação de borda/IoT), pois os ensembles gerados costumam ser pesados e lentos para inferência. Além disso, em cenários onde os dados são majoritariamente ruídos, não estruturados (textos muito complexos, áudios) ou exigem engenharia de características (Feature Engineering) baseada em regras de negócio altamente específicas, o AutoML convencional tabula de forma ineficiente, desperdiçando processamento.

## **10. Após realizar a atividade, qual ferramenta a equipe recomendaria para iniciantes? Justifique.**

De forma unânime, a equipe recomendaria o MLJAR para iniciantes não na área de data science, mas ao domínio de AutoML. Para um iniciante, a etapa mais desafiadora da Ciência de Dados não é treinar o modelo, mas entender como o modelo tomou suas decisões. O fato do MLJAR gerar explicações visuais (XAI), leaderboards legíveis e matrizes automáticas fornece um feedback educacional imediato. Ele permite que o iniciante estude a importância dos atributos e a dinâmica do modelo, atuando quase como um tutor interativo, enquanto o FLAML opera de forma mais silenciosa e voltada à performance pura, ideal para usuários mais maduros no ecossistema de dados.

## ATRIBUTOS DO DATASET E EVIDÊNCIAS:

### Nova Configuração de atributos:

Os atributos extraídos para as ferramentas de automl se dão por:

1. **age (idade)** - variável numérica inteira.
2. **gender (gênero)** - variável categórica numérica (0 = male; 1 = female).
3. **daily\_social\_media\_hours (horas diárias nas redes sociais)** - variável numérica contínua.
4. **platform\_usage (uso da plataforma)** - variável categórica numérica.
5. **sleep\_hours (horas de sono)** - variável numérica contínua.
6. **screen\_time\_before\_sleep (tempo de tela antes de dormir)** - variável numérica contínua.
7. **academic\_performance (desempenho acadêmico)** - variável numérica contínua.
8. **physical\_activity (atividade física)** - variável numérica contínua.
9. **social\_interaction\_level (nível de interação social)** - variável categórica numérica (low = 0, medium = 1, high = 2).
10. **addiction\_level (nível de dependência)** - variável numérica inteira.
11. **depression\_label (indicador de depressão)** - variável categórica/binária (TARGET).

### GitHub do Projeto:

[https://github.com/PedroYutaroUEA/depression-prediction\\_autoML](https://github.com/PedroYutaroUEA/depression-prediction_autoML)